ITC 6050 — Week 2 Lab

Designing, Normalising, and Optimising a Relational Schema

> **Estimated time:** 90 minutes (lab + stretch). **Goal:** Practise the full design loop — sketch a conceptual model, translate it to a physical PostgreSQL schema, fix a deliberately broken schema by normalising it to 3NF, then load realistic data and prove you can make it faster with indexes.
> **Submission:** Push your work to a new GitHub repository named `itc6050-w2-lab-<yourname>` and submit the URL on Blackboard.

What you'll build

Across four exercises, you will design and tune a small e-commerce schema:

```
1.  Sketch    →   Conceptual model    (entities & relationships, no types)
2.  Refine    →   Logical model       (keys, attributes, types)
3.  Build     →   Physical schema      (PostgreSQL DDL)
4.  Fix       →   Normalise a bad schema to 3NF
5.  Tune      →   Indexes & EXPLAIN ANALYZE
```

By the end you'll have a clean, well-typed schema in Postgres, a normalised version of a deliberately broken table, and concrete before/after numbers showing how indexes change query plans.

Before you begin

You need everything from Week 1:

- Postgres running in Docker (`docker compose up -d` from your W1 lab folder works fine — or follow the same steps in a fresh folder)
- Your Python virtual environment activated
- Optional but recommended: **DBeaver** or **psql** for ad-hoc queries

> **Tip:** Keep three windows open — VS Code (your `.sql` files), DBeaver/psql (running queries), and your browser at **dbdiagram.io** (sketching).

Step 0 — Set up your project *(5 min)*

```
mkdir itc6050-w2-lab && cd itc6050-w2-lab
git init
mkdir sql diagrams
touch README.md sql/01_physical_schema.sql sql/02_normalisation.sql
sql/03_load_data.sql sql/04_indexing.sql
```

Push to a new GitHub repo named `itc6050-w2-lab-<yourname>` (same flow as W1).

Reuse the Postgres container from Week 1, but create a **fresh database** so this week's work doesn't collide:

```
docker exec -it itc6050_pg psql -U itc6050 -d postgres -c "CREATE DATABASE shop_lab;"
```

You'll connect to `shop_lab` (not `lab`) for the rest of this exercise.

Step 1 — Sketch a conceptual model *(15 min)*

You're designing a tiny **e-commerce site**. Real customers buy real products, with addresses, categories, and orders that contain multiple items.

Open dbdiagram.io (no signup required) and create a new diagram. Sketch the entities and relationships — **no types, no keys yet**. Just nouns and verbs.

You should end up with **at least** these entities; add more if your design calls for them:

- Customer
- Address
- Category
- Product
- Order
- OrderItem

Use Crow's-Foot notation (dbdiagram defaults to it). Think about:

- A customer can have **many** addresses (shipping, billing). Can an address be shared?
- A product belongs to **one** category — or could it belong to many?
- An order has **many** order items; each order item references **one** product.
- What's the difference between an **order** and an **order item**? Why do we need both?

Save your diagram as a PNG and drop it in `diagrams/conceptual.png` in your repo.

> 🎯 **Concept check:** Why is `OrderItem` a separate entity rather than just a `product_id` column on `Order`? Write a one-sentence answer in your `README.md`.

> 💪 **Stretch:** Add a `Review` entity (a customer reviews a product) and a `Discount` entity (applies to a category or a specific product, time-bound).

Step 2 — Translate to a logical model *(15 min)*

Stay in dbdiagram.io. Now **add attributes, primary keys, and foreign keys** — but stay vendor-neutral. No `BIGINT`, no `JSONB`, just generic types: `int`, `varchar`, `date`, `decimal`, `boolean`.

Here's a starter for one entity to show the syntax dbdiagram expects:

```
Table customer {
  customer_id   int        [pk]
  email         varchar    [unique, not null]
  first_name    varchar    [not null]
  last_name     varchar    [not null]
  created_at    datetime   [default: `now()`]
}
```

Do the same for every entity from Step 1. Add foreign keys with a `Ref:` line:

```
Ref: order_item.order_id   > order.order_id
Ref: order_item.product_id > product.product_id
```

Save the DBML source to `diagrams/logical.dbml` and an exported PNG to `diagrams/logical.png`.

> 🎯 **Concept check:** Why does the logical model deliberately avoid Postgres-specific types like `BIGINT` and `JSONB`? Add your answer to `README.md`.

> 💪 **Stretch:** Identify all the relationships that are 1-to-many vs. many-to-many. For each many-to-many, name the join table.

Step 3 — Build the physical schema in PostgreSQL *(15 min)*

Time to make it real. Open `sql/01_physical_schema.sql` and translate your logical model into PostgreSQL.

A starter — fill in the rest of the tables yourself:

```
-- sql/01_physical_schema.sql
DROP SCHEMA IF EXISTS shop CASCADE;
CREATE SCHEMA shop;
SET search_path TO shop;

CREATE TABLE customer (
  customer_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email        VARCHAR(255) NOT NULL UNIQUE,
  first_name   VARCHAR(80)  NOT NULL,
  last_name    VARCHAR(80)  NOT NULL,
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE TABLE address (
  address_id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id  BIGINT NOT NULL REFERENCES customer ON DELETE CASCADE,
  line1        VARCHAR(120) NOT NULL,
  city         VARCHAR(80)  NOT NULL,
  postcode     VARCHAR(20)  NOT NULL,
```

```
  country      CHAR(2)      NOT NULL,
  is_default   BOOLEAN      NOT NULL DEFAULT FALSE
);

-- TODO: category, product, "order", order_item
-- HINT: "order" is a reserved word in SQL — quote it or rename to orders.
```

Your job: add `category`, `product`, `orders` (use the plural to avoid the reserved word), and `order_item`.

Make sure every table has:

- A surrogate primary key (`BIGINT GENERATED ALWAYS AS IDENTITY`)
- Appropriate `NOT NULL` constraints
- Foreign keys with deliberate `ON DELETE` behaviour
- At least one `CHECK` constraint somewhere (e.g. `total >= 0`, `quantity > 0`, `country ~ '^[A-Z]{2}$'`)

Run it:

```
docker exec -i itc6050_pg psql -U itc6050 -d shop_lab <
sql/01_physical_schema.sql
```

Verify the schema:

```
docker exec -it itc6050_pg psql -U itc6050 -d shop_lab \
  -c "\dt shop.*"
```

You should see all six tables.

> 🎯 **Concept check:** You used `ON DELETE CASCADE` on `address.customer_id` — why is that probably *the wrong choice* for `order_item.order_id`? (Hint: think audit trail.)

> 💪 **Stretch:** Add a `CHECK` constraint that enforces email looks roughly like an email (`email ~ '^[^@]+@[^@]+\.[^@]+$'`). Add a partial index on `address (customer_id) WHERE is_default = TRUE`.

Step 4 — Normalise a bad schema to 3NF *(20 min)*

A junior engineer hands you the following table and asks you to "make it better." It works — but it's a mess.

Save this in `sql/02_normalisation.sql` and run it on `shop_lab`:

```
-- sql/02_normalisation.sql
CREATE SCHEMA IF NOT EXISTS bad;
SET search_path TO bad;

CREATE TABLE sales_record (
  sale_id            BIGINT,
  sale_date          DATE,
  customer_name      VARCHAR(160),
  customer_email     VARCHAR(255),
  customer_phones    VARCHAR(255),    -- comma-separated! e.g. '555-1212,
555-3434'
  customer_city      VARCHAR(80),
  customer_country   VARCHAR(60),
  product_name       VARCHAR(120),
  product_category   VARCHAR(60),
  product_price      NUMERIC(10,2),
  quantity           INT,
  line_total         NUMERIC(10,2)
);
```

4a — Identify the violations

Add comments to your `sql/02_normalisation.sql` file answering:

1. **1NF violation** — which column has multi-valued data? Quote the offending column.
2. **Functional dependency 1** — `product_name → product_category, product_price`. Why does this break 3NF?
3. **Functional dependency 2** — `customer_email → customer_name, customer_city, customer_country`. Why does this break 3NF?
4. Give one concrete example of an **update anomaly** that this schema enables. (E.g. "If a product's price changes, …").

4b — Decompose to 3NF

Below your comments, write the **decomposed schema**. You'll need at least:

- `customer (customer_id, email, name, city, country)` — with `email UNIQUE`
- `customer_phone (customer_id, phone)` — many phones per customer
- `product (product_id, name, category, unit_price)`
- `sale (sale_id, customer_id, sale_date)`
- `sale_item (sale_id, product_id, quantity, unit_price_at_sale, line_total)`

Notice the `unit_price_at_sale` column: prices change over time, so the sale must record the price *at the moment of sale*, not look it up dynamically. That's a common 3NF debate point.

Write proper `CREATE TABLE` statements with primary keys, foreign keys, and constraints, all in `bad` schema (or `clean` — your call).

> 🎯 **Concept check:** A colleague argues the `unit_price_at_sale` column is "denormalising — it duplicates `product.unit_price`." Are they right? Defend your design in two sentences.

> 💪 **Stretch:** Take it to **BCNF**. Is your 3NF schema already in BCNF? If yes, prove it. If no, find the offending functional dependency and decompose further.

Step 5 — Load realistic data *(10 min)*

Indexing only matters when there's enough data to slow things down. We'll generate **synthetic but realistic** data using PostgreSQL's `generate_series` — no Python or external tools needed.

Save this as `sql/03_load_data.sql`:

```
-- sql/03_load_data.sql
SET search_path TO shop;

-- 50 categories
INSERT INTO category (name)
SELECT 'Category ' || g
FROM generate_series(1, 50) g;

-- 1,000 products
INSERT INTO product (category_id, name, unit_price)
SELECT
  ((g - 1) % 50) + 1,
  'Product ' || g,
  ROUND((random() * 200 + 5)::numeric, 2)
FROM generate_series(1, 1000) g;

-- 10,000 customers
INSERT INTO customer (email, first_name, last_name)
SELECT
  'cust' || g || '@example.com',
  'First' || g,
  'Last' || g
FROM generate_series(1, 10000) g;

-- 100,000 orders, spread over the last 2 years
INSERT INTO orders (customer_id, order_date, status, total)
SELECT
  ((g - 1) % 10000) + 1,
  NOW() - (random() * INTERVAL '730 days'),
  (ARRAY['new','paid','shipped','delivered','cancelled'])
[ceil(random()*5)],
  ROUND((random() * 500 + 10)::numeric, 2)
FROM generate_series(1, 100000) g;

-- 500,000 order items (avg 5 per order)
INSERT INTO order_item (order_id, product_id, quantity, unit_price_at_sale)
SELECT
  ((g - 1) % 100000) + 1,
  ((g - 1) % 1000) + 1,
  ceil(random() * 5)::int,
  ROUND((random() * 200 + 5)::numeric, 2)
```

```
FROM generate_series(1, 500000) g;

-- Refresh planner statistics so EXPLAIN ANALYZE has good estimates
ANALYZE;
```

> **Note:** You may need to adjust column names/types if you used different ones in Step 3 (`orders.order_date` vs `orders.created_at`, `category(name)` vs `category(category_name)`, etc.). That's intentional — making the script run *is* part of the exercise.

Run it:

```
docker exec -i itc6050_pg psql -U itc6050 -d shop_lab <
sql/03_load_data.sql
```

Verify:

```
SELECT
  (SELECT COUNT(*) FROM shop.customer)   AS customers,
  (SELECT COUNT(*) FROM shop.product)    AS products,
  (SELECT COUNT(*) FROM shop.orders)     AS orders,
  (SELECT COUNT(*) FROM shop.order_item) AS items;
```

You should have **10 000 / 1 000 / 100 000 / 500 000**.

Step 6 — Indexes & EXPLAIN ANALYZE *(20 min)*

Now we make it fast — and prove it.

6a — Run baseline queries (no extra indexes yet)

Save this as `sql/04_indexing.sql`. We'll add to it as we go.

```
-- sql/04_indexing.sql
SET search_path TO shop;

-- Q1: Look up a customer by email
EXPLAIN ANALYZE
SELECT *
FROM   customer
WHERE  email = 'cust5000@example.com';

-- Q2: All orders for a given customer in date order
EXPLAIN ANALYZE
SELECT order_id, order_date, total
FROM   orders
```

```
WHERE  customer_id = 5000
ORDER  BY order_date DESC;

-- Q3: Top 10 products by revenue in the last 90 days
EXPLAIN ANALYZE
SELECT  p.name,
        SUM(oi.quantity * oi.unit_price_at_sale) AS revenue
FROM    order_item oi
JOIN    orders     o USING (order_id)
JOIN    product    p USING (product_id)
WHERE   o.order_date >= NOW() - INTERVAL '90 days'
GROUP   BY p.name
ORDER   BY revenue DESC
LIMIT   10;
```

Run these one at a time. **For each query, copy the plan and the *Execution Time* line into your README.** You should see things like:

- `Seq Scan on customer` — Postgres reads every row to find one email
- `Sort` followed by `Limit` instead of using an index
- Total execution times in the **tens or hundreds of milliseconds**

6b — Add indexes and re-measure

Append the following, run, then re-run the three queries above:

```
-- Speed up customer-by-email lookup
CREATE INDEX idx_customer_email ON customer (email);

-- Speed up "orders for a customer, recent first"
CREATE INDEX idx_orders_customer_date ON orders (customer_id, order_date
DESC);

-- Speed up the date-range scan in Q3
CREATE INDEX idx_orders_date ON orders (order_date);

-- Speed up the join + aggregation in Q3
CREATE INDEX idx_order_item_order   ON order_item (order_id);
CREATE INDEX idx_order_item_product ON order_item (product_id);

ANALYZE;
```

Re-run **Q1, Q2, Q3** and copy the new plans + Execution Times into your README, side-by-side with the baseline. You should see:

- `Index Scan` or `Index Only Scan` instead of `Seq Scan`
- Execution times **10–1000× faster** depending on the query

6c — Reflect

In your `README.md`, answer **all four**:

1. Which query saw the biggest speed-up? Why?
2. Look at Q2's plan — did Postgres also use the index for ordering, or did it sort separately? How can you tell from the plan?
3. We added `idx_order_item_product` but never queried by `product_id` alone in Q1–Q3. Why is it still useful?
4. **Cost of indexes:** what trade-off did we make by adding all these indexes? Name two operations that are now slightly slower.

> 💪 **Stretch challenges — pick one:**
>
> 1. **Partial index:** create one that only indexes orders `WHERE status = 'paid'` and prove it's smaller than the full index using `\di+ idx_*` in psql.
> 2. **Composite ordering:** can you create a single composite index that satisfies *both* Q2 and a query like "all orders for customer 5000 with status='paid'"? Test it.
> 3. **GIN on JSONB:** add a `metadata JSONB` column to `orders`, populate it with random JSON for half the rows, and create a GIN index. Query `WHERE metadata @> '{"source":"web"}'`.
> 4. **Find a dead index:** use `pg_stat_user_indexes` to find an index you created that doesn't get used. Drop it and explain why.

What you experienced

In this lab you walked through the entire schema-design loop:

Step	What you did	Why it matters
Conceptual	Drew entities & relationships	Captures business meaning — no tech yet
Logical	Added keys, attributes, generic types	Vendor-neutral; talks to architects
Physical	Wrote PostgreSQL DDL with constraints	Real, runnable, deployable
Normalise	Decomposed a bad schema to 3NF	Removes redundancy, prevents anomalies
Index & tune	Added indexes, compared EXPLAIN plans	Made queries 10–1000× faster — and you can prove it

This is the day-to-day work of a data engineer designing OLTP systems. From Week 4 onwards we'll add a warehousing layer on top — but the discipline you practised today is the foundation everything else stands on.

Submission checklist

Your repo `itc6050-w2-lab-<yourname>` must contain:

- ☐ `diagrams/conceptual.png` — your hand-sketch or dbdiagram export

- [ ] `diagrams/logical.dbml` and `diagrams/logical.png`
- [ ] `sql/01_physical_schema.sql` — runnable, with all six tables, constraints, and your `CHECK`s
- [ ] `sql/02_normalisation.sql` — the bad table, your written analysis (in comments), and the decomposed 3NF schema
- [ ] `sql/03_load_data.sql` — possibly modified to match your column names
- [ ] `sql/04_indexing.sql` — baseline queries, indexes, and re-measure
- [ ] `README.md` — concept-check answers, Q1–Q3 before/after plans + execution times, the four reflection questions answered
- [ ] `.gitignore` — same as Week 1

Push your final commit and submit the **GitHub repository URL** on Blackboard before next week's lab.

Troubleshooting appendix

`CREATE TABLE order` fails with a syntax error. `ORDER` is a reserved SQL keyword. Either name your table `orders` (recommended) or quote it as `"order"` everywhere — *every* time.

Foreign key fails: "violates foreign key constraint." Your insert order matters. You must populate parent tables (`customer`, `product`) before child tables (`orders`, `order_item`). Check the order of `INSERT`s in `03_load_data.sql`.

EXPLAIN ANALYZE shows `Seq Scan` even after I added an index. Three common causes: (1) you forgot to run `ANALYZE` after adding the index — Postgres uses outdated statistics; (2) the table is small enough that a seq scan is genuinely faster; (3) the WHERE clause uses a function or implicit cast that disables the index (e.g. `WHERE LOWER(email) = ...`).

Execution times barely change. Most likely you're querying for a value that doesn't exist or doesn't match many rows — both work fine on tiny result sets even without an index. Try `customer_id = 5000` (which exists) rather than `customer_id = 999999`.

Anything else. Post in the course discussion board with the exact error and what you tried — peer help encouraged.

End of Week 2 lab. See you next week for SQL & NoSQL.